

MDP 정식화 (Markov Decision Processes)

MDP의 다섯 요소와 마르코프 성질 · 누적 보상과 할인율 · 가치함수와 벨만 기대방정식

Machine Learning 2 · 3주차

한국과학영재학교 (KSA)

Chapter 3

이번 주차 개요

1950년대 벨만(Richard Bellman, 1957)은 “차원의 저주”라는 용어를 만들며 **동적계획법**을 고안. 핵심 통찰 — **복잡한 문제를 한 번에 풀지 말고, 작은 부분 문제로 쪼개 그 답을 재귀적으로 조합하라** — 가 지금 강화학습 이론 전체의 뼈대.

- Chapter 2의 밴딧은 **상태가 정확히 하나뿐인, 전이도 없는 특수한 MDP**.
- 이번 장은 밴딧에 **상태**와 **전이**를 더해, “지금 행동이 다음 상태를 바꾸고, 그 상태가 미래 보상을 결정한다”는 순환을 수학적으로 다룬다.

이 챕터를 마치면

- MDP의 다섯 요소 (S, A, P, R, γ) 를 한 줄로 쓰고, **마르코프 성질**이 무엇을 요구하는지 판단한다.
- 할인된 누적 보상(리턴) G_t 를 손으로·코드로 계산하고, γ 의 두 얼굴(스려 자치 + 선택 벅스)을

세 차시

- ① **3.1 MDP의 다섯 요소와 마르코프 성질** – “게임의 규칙 전체”를 다섯 요소로 닫고, 3칸 보드게임 MDP를 통째로 써본다.
- ② **3.2 누적 보상, 할인을, 그리고 실습** – 리턴 G_t 계산, **유효 시계**, “ γ 가 목표를 바꾼다”를 확인한다.
- ③ **3.3 가치함수와 벨만 기대방정식** – V^π, Q^π 정의, 무한합을 한 스텝 재귀식으로 압축하고, 반복 대입을 예고한다.

이 “문법”을 확실히 잡아야, Ch. 4 정책평가 · Ch. 6 Q-러닝 · Ch. 9 DQN 손실함수가 사실 **동일한 재귀식(벨만방정식)**을 각각 다른 방식으로 계산. 함수형은 게임을 끝내보아야 볼 수 있다.

3.1 MDP의 다섯 요소와 마르코프 성질

이 절에서 무엇을 만들 것인가

이 절은 두 가지를 만든다.

- 1 강화학습 문제를 **수학적으로 한 줄에** 쓰는 법 — MDP는 다섯 가지 요소 ($\mathcal{S}, \mathcal{A}, P, R, \gamma$) 하나로 “게임의 규칙 전체”를 담은 형식.
- 2 그 형식이 성립하려면 필요한 **마르코프 성질**의 정확한 의미와, 실제로는 언제 깨지는지(깨졌을 때 어떻게 다시 성립 시키는가).

이 두 가지는 앞으로 16개 챕터에서 매일 쓰게 될 **언어의 사전**이다.

- 벨만방정식(3.3), 정책평가(4.1), Q-러닝(6)의 모든 수식이 결국 $\mathcal{S}, \mathcal{A}, P, R$ 위에서 쓰인 것.
- 여기의 표기가 헛갈리면 뒤의 모든 것이 흐릿해진다.

MDP의 다섯 요소

MDP는 다섯 가지로 정의된다: $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$

요소를 한 줄로	의미
$\mathcal{S}$	가능한 상태 들의 집합
$\mathcal{A}$	가능한 행동 들의 집합
$P(s' s, a)$	s 에서 a 를 했을 때 s' 로 전이 될 확률
$R(s, a)$	s 에서 a 를 했을 때 받는 즉시 보상
$\gamma \in [0, 1)$	할인율 (discount factor)

Chapter 2의 밴딧과 비교

밴딧에는 $\mathcal{S}$ 도 P 도 **없었다**(팔을 당겨도 “다음 상태”가 없었다). MDP는 지금 한 행동이 다음 상태 $P(s'|s, a)$ 를 통해 **미래 전체**에 영향을 준다는 사실을 명시적으로 담는다.

“다섯 개면 충분한가?” — 상태가 화폐

MDP의 정의를 읽으면 “상태”가 모든 정보의 **화폐(currency)**로 등장한다.

- 보상은 (s, a) 에서, 다음 상태는 (s_t, a_t) 에서 결정된다.
- 즉 **상태만 알면 앞으로 일어날 것의 전체 확률법칙이 결정된다.**

이 문장이 바로 **“마르코프 성질”**이라고 부르는 것의 **정확하고 강한 형태**(다음 섹션에서 정식화).

다섯 요소가 “게임의 규칙 전체”를 담는다는 뜻

초기 상태 분포는 **알고리즘에 필요한 정보가 아니다** — 학습은 어떤 시작점에서든 똑같이 진행되며, 그 점은 연습문제 4에서 확인한다.

역사: 이름이 붙은 두 수학자, 반세기의 드라마

마르코프(Andrei Markov, 1856-1922)

- 확률론과 수론을 오가던 러시아 수학자.
- 1906년 논문: 당시 확률의 대명사 **베르누이 법칙** (독립 반복 시행의 상대빈도 $\rightarrow$ 확률)의 왕좌를 지키던 확률론에 **의존하는 시행**까지 포괄.
- “이번 결과가 직전 결과만 기억하고 이전은 잊는” 의존성(= 지금의 마르코프 체인)에서도 베르누이 법칙이 성립함을 보임.

의존성이 들어와도 법칙이 살려면 그 의존성은 **1단계만**이어야 한다는 발견이, 120년 뒤 강화학습에서 “상태 하나만 기억하면 된다”는 설계 원칙으로 직접 이어진다.

벨만(Richard Bellman, 1920-1984)

- 2차전지 중 미공군 폭격 편성 문제를 풀면서 발견: **최적 전략은 부분 문제의 최적 전략을 재귀적으로 조합할 수 있다**(최적성의 원리).
- 1957년 저서 Dynamic Programming에서 세운 프레임워크가 바로 MDP의 정식화.

흥미로운 우연: 마르코프 성질 논문(1906)과 MDP 정식화(1957) 사이는 50년, 그런데 두 사람의 문제는 **정확히 같은 문제의 양쪽 끝**이다.

같은 문제의 양쪽 끝

마르코프: “이 의존 구조가 어떤 **확률법칙**을 결정하는가”(기술, description)

벨만: “그 법칙 위에서 **최적 행동**을 어떻게 찾는가”(최적화, optimization)

이번 한기의 구조(이번 장 = 전신화 $\rightarrow$ Ch 4-6 = 최적화)가 이 드라마의 둘째막

한 편의 MDP를 통째로 쓰기: 3칸 보드게임

추상적으로 다섯 요소를 나열하는 것만으로는 체감되지 않는다. 아주 작은 MDP 하나를 **통째로** 써본다.

설정: 상태 0, 1, 2의 보드게임. 0에서 시작, 매 스텝 “오른쪽으로 한 칸” 또는 “자리참기”를 고를, 상태 2에 도착하면 **에피소드 종료**. 이동은 확률적: “오른쪽”을 골라도 0.3의 확률로 제자리에 미끄러진다.

- $\mathcal{S} = \{0, 1, 2\}$ — 보드의 세 칸. 2는 **터미널 상태**(terminal state, 게임 종료 상태).
- $\mathcal{A} = \{\text{right, wait}\}$ — 두 행동.
- $\gamma = 0.9$.

이 다섯 개(5-tuple)가 쓰이는 순간, 이 게임의 “규칙”은 **완전히** 정해졌다. 어떤 시작점에서든, 어떤 정책(상태마다 행동을 고르는 규칙)으로 놀든, 받을 보상 시퀀스의 **전체 확률분포**가 이 다섯 요소만으로 계산된다 — 게임에 “보이지 않는 상태”가 없어야 한다는 전제(= 마르코프 성질)를 붙이면 정확하다. 다음 장에서 벨만방정식으로 $V^\pi(s)$ 를 계산하면, 여기 쓴 P 와 R 이 그대로 수식에 대입된다.

3칸 보드게임: 전이확률 P 와 보상 R

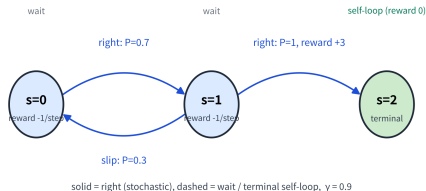
전이확률 P

상태	행동	s' (확률)
0	right	1 (0.7), 0 미끄러짐 (0.3)
0	wait	0 (1)
1	right	2 (1)
1	wait	1 (1)
2 (터미널)	(모든)	2 (1) self-loop

보상 R

- 터미널로 들어갈 전이(1에서 right): **+3**
- 그 외 모든 스텝: **-1** (한 스텝 움직이는 데 드는 “시간 벌금”)

3-state board-game MDP — the entire ruleset is captured by (S, A, P, R, γ)



3상태 보드게임 MDP — 상태 0에서 시작,
right(실선)/wait(점선) 전이, 미끄러짐 확률 0.3,
터미널 2에 진입 시 +3(그 외 매 스텝 -1).

각 요소 하나씩 (1/2): $\mathcal{S}$, $\mathcal{A}$

$\mathcal{S}$, 상태 공간

- 상태는 “에이전트의 관점에서의 **세계의 완전한 기술**”이어야 한다.
- CartPole의 상태 $[x, \dot{x}, \theta, \dot{\theta}]$ 가 4차원인 이유: 위치 두 개만 알고 속도 두 개를 모르면 “막대가 아직 기울어지고 있는가, 멈춰 있는가”를 분간할 수 없다.
- **상태에 빠진 정보는 에이전트에게 영구히 사라진 정보**다 — 보상이나 전이를 통해 돌아올 통로가 없기 때문.
- 이 “상태 설계”가 MDP를 세울 때 **가장 많은 시간이 드는 일**. 연속 상태 공간도 흔하다(CartPole처럼 $\mathcal{S} \subset \mathbb{R}^n$ 인 경우) — 표가 아니라 신경망으로 함수를 표현하게 된다(Ch. 9–11).

$\mathcal{A}$, 행동 공간

- 이산: CartPole의 left/right, FrozenLake의 4방향.
- 연속: Pendulum의 $[-2, 2] \subset \mathbb{R}$ (힘의 크기 그 자체를 고른다).
- 이산/연속의 차이는 곧 “어떤 함수를 표현할가”의 문제 — 이산이면 상태 $\times$ 행동 크기의 **표**, 연속이면 **연속 출력의 신경망**(Ch. 9 vs Ch. 10–11의 분기가 여기서 나온다).

각 요소 하나씩 (2/2): P, R, γ

P , 전이확률

- $P(s'|s, a)$ 는 “이 게임의 **물리 법칙**”이다.
- Gymnasium 환경에서는 이 함수가 `step()` 함수의 내부 구현으로 감춰져 있고, 우리는 **샘플**(실제 전이)만 관찰할 수 있다.
- **모델 기반**(Ch. 4-5 동적계획법): P 를 (추정해서) 명시적으로 써서 계획. **모델 없는**(Ch. 6-11): P 를 전혀 쓰지 않고 경험(실제 전이 샘플)에서만 가치를 배운다.
- 같은 P 라도 “정식적 표”로 줄 것이냐 “샘플 공급 원”으로 둘 것이냐가 알고리즘의 가족을 갈라놓는 **첫 번째 갈림길**.

R , 보상

- 수학적으로 더 정확한 표기는 **전이** $R(s, a, s')$ 에 붙는 것(같은 (s, a) 에서도 다음 상태가 다르면 보상이 다를 수 있음 — “도착했다/아직이다”는 s' 를 봐야 알 수).
- 보상의 **단위.스케일**은 알고리즘에 직접 영향을 준다: 10배 키우면 가치함수 전체가 10배가 되지만 **최적 정책은 변하지 않는다**(연습문제 5).
- 그러나 **상태/행동 사이에서 불균일**하게 들쭉한 스케일 은 조심해야 한다(Ch. 5에서 탐험 균형이 깨지는 사례).

 **할인율:** $\gamma < 1$ 은 “에이전트가 영원히 살지 않는 세계”를 뜻하며, V^π, Q^π 가 **유계 함수**로 존재하게 하는 수학적 필요조건 (연습문제 2.2에서 보자)

마르코프 성질 (Markov property)

정의

마르코프 성질: 다음 상태는 오직 **현재** 상태와 행동에만 의존한다 — 어떻게 지금 상태에 도달했는지(과거 전체 이력)는 상관없다:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0) = P(s_{t+1} \mid s_t, a_t)$$

- **예:** 체스판의 현재 배치만 알면, 거기까지 어떤 수순을 거쳐왔는지는 다음 수를 정하는 데 필요 없다.
- 이 성질 덕분에 “지금까지의 전체 이력”이 아니라 “현재 상태” **하나만 기억하면 충분**해진다 — 뒤에서 배울 알고리즘들이 상태 하나만 보고 결정을 내릴 수 있는 이유가 여기서 나온다.

“상관없다”가 확률적으로 정확히 무엇을 뜻하는가 — 조건부 확률의 정의와 사건의 독립성으로부터:

$$P(s_{t+1} = s' \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} = s' \mid s_t, a_t)$$

가 **모든** 과거 이력과 **모든** 다음 상태 s' 에 대해 성립할 때, s_t 가 (a_t 를 준 상태에서) **마르코프 성질**을 가진다고 한다.

왜 이 성질이 알고리즘의 모든 것을 결정하는가

마르코프 성질이 “좋은 가정”을 넘어서 **필요 불가결**인 이유. **성질이 없다면:**

상태가치가 정의되지 않는다

- $V^\pi(s)$ 는 “상태 s 에서 시작”의 리턴 기댓값.
- “같은 s ”에 과거 이력이 **다른** 두 경로가 존재하고 그 미래 분포가 다르면, s 만으로 인자화한 하나의 함수 $V(s)$ 로 그 둘을 동시에 설명할 수 없다.
- 같은 자리에 “두 개의 다른 가치”가 공존하는데, 표 기반 알고리즘은 **한 칸에 한 값**만 쓸 수 있다.

학습이 수렴하지 않는다

- 같은 s 를 매번 **다른 “진짜 상황”**의 대리인으로 쓰면, 추정치는 서로 다른 답을 향해 계속 튀어 다닌다.
- Ch. 6-11의 모든 수렴 증명은 “같은 s 는 같은 미래 분포를 갖는다”는 **이 동일성** 위에서 있다.

반대로: 성질이 성립하면

상태 하나가 **과거 전체를 압축하는 무손실 요약**이 된다 — 통계학의 표현을 빌리면, 상태는 미래에 대한 과거 이력의 **충분 통계량(sufficient statistic)** 역할을 한다. 이 성질 덕분에 “기억의 길이”가 문제에서 빠져나가고, **현재 관측** 하나로 의사결정이 닫힌다.

마르코프 성질이 성립하지 않을 때: hidden state와 재설계

현실은 대부분 **부분관측**(partially observed)이다 — 에이전트가 보는 “관측”이 진짜 상태의 완전한 투영이 아니다.

대표 예: 자율주행 — 카메라 한 장만 관측으로 쓰면 “지금 얼마나 빠르게 움직이고 있었는가”라는 정보가 빠져, **같은 화면**에서 “속도가 높은 상황”과 “속도가 낮은 상황”이 구별되지 못한다 — 같은 s_{obs} 임에도 미래 분포가 **다르다** $\Rightarrow$ 성질이 깨진다.

상태 재설계의 두 가지 표준 전략

전략 1: 상태에 정보를 직접 넣는다 — 최근 k 프레임 $(o_t, o_{t-1}, \dots, o_{t-k+1})$ 을 하나의 “확장 상태”로 묶어 s_t 로 재정의. k 가 충분하면 **속도 정보가 (프레임 차이로) 복원**되어 성질이 다시 성립. 해석은 쉽지만 “ k 를 몇으로 할지”라는 **손으로 고르는 하이퍼파라미터**가 생긴다.

전략 2: 학습된 은닉 상태로 요약한다 — 신경망(예: RNN의 은닉 상태 h_t , Ch. 11)이 과거 관측 시퀀스를 h_t 로 압축하고 (o_t, h_t) 를 상태로 쓴다. “요약에 충분한 정보를 담았는지”는 수학적으로 검증할 수 없고 **경험적으로** 판단. 이것이 **부분관측 MDP(POMDP)** 라는, 이번 학기에서 직접 다루지는 않지만 **이름은 꼭 알아 둘** 문제 영역.

핵심

마르코프 성질은 “자연이 주는 성질”이 아니라, **우리가 상태를 어떻게 정의하는가의 설계 선택**이다. 성질이 “깨져” 보이면, 그건 우리의 상태 정의가 **부족하다는 신호**의 뿐. 강하게 스스로 자체를 포기해야 할 이유가 아니다.

확인 문제: MDP인가, 마르코프인가? (1/2)

네 시나리오 각각에 대해 (a) MDP로 모델링할 수 있는가, (b) 가능하다면 $\mathcal{S}, \mathcal{A}, P, R$ 을 한 줄씩 쓰라:

- (a) 5층 건물의 엘리베이터 호출 제어

- ▶ 답: **MDP** — $\mathcal{S} = (\text{각 층의 상/하 호출 버튼 상태 } 2^{10} \text{ 조합}) \times (\text{엘리베이터 현재 층 } 5) = 5120\text{개}$ 이하, $\mathcal{A} = \{up, down, open\}$, $P = \text{버튼 상태} + \text{층 이동}$, $R = -1(\text{매 스텝 벌금})$ 또는 “누락 호출” 페널티.
- ▶ 마르코프 성질은 **성립**(현재 호출 상태가 미래의 모든 것을 결정).

- (b) “오늘 기분”을 몰라서, 어제 몇 시간 폰을 봤는 것만 관측으로 가지는 추천 문제

- ▶ 답: **마르코프 성질이 깨진** 문제 — “기분”이 **hidden state**.
- ▶ 어제 폰 사용 시간만으로는 같은 관측 아래 “피곤한 날”과 “재밌는 날”을 구별 못 해 **추천의 결과 분포가 달라진다**.
- ▶ 해결: 최근 며칠의 관측을 확장 상태로 묶거나 (전략 1), 사용자 모델의 은닉 상태로 요약(전략 2).

확인 문제: MDP인가, 마르코프인가? (2/2)

- (c) 슬롯머신(Chapter 2의 밴딧)

- ▶ 답: **MDP이지만 특수한 경우** — $\mathcal{S} = \{s_0\}$ (상태 1개), $P(s_0|s_0, a) = 1$ for all a .
- ▶ 밴딧은 “**전이 없는 MDP**”로, 이 절의 모든 정의가 **퇴화된 형태로** 적용(FAQ 참고).

- (d) 체스에서 다음 수를 고르는 문제(중반)

- ▶ 답: **MDP** — $\mathcal{S}$ = 합법적인 모든 판 배치(크기 $\sim 10^{40}$ 이상), $\mathcal{A}$ = 그 배치의 합법 수, P = 행동 후 판 배치(**결정적**이라 확률 1), R = 매 스텝 0, 체크메이트 시 ± 1 .
- ▶ 체스는 마르코프 성질이 **정확히** 성립하는 **희귀한 예** — “어떤 수순을 거쳐왔는지”가 다음 선택지에 영향을 주지 않으므로.

네 문항의 공통 기준

“**현재 상태(관측)만 알면, 앞으로 일어날 것의 전체 확률분포가 결정되는가?**” 아니라면 hidden state가 있다는 뜻이고, 상태 재설계(전략 1 또는 2)가 필요하다.

자주 하는 실수 (1/2): “관측”과 “상태”를 혼동

실습 코드에서 **가장 흔한 에러**: `obs`(환경이 돌려주는 관측)를 $\mathcal{S}$ 의 원소인 **상태**로 그대로 쓰고 마는 것. 두 개는 같은 환경에서 **같은 길이**를 가진 벡터일 수 있지만, **역할이 다르다**.

관측(observation)

- 에이전트가 **볼 수 있는** 것.
- 마르코프 성질을 **보장하지 않는다** — Partially Observed MDP에서는 $o_t \neq s_t$ 다.
- 이번 학기의 Gymnasium 환경(CartPole, FrozenLake, Pendulum)은 **완전관측**이라 `obs`를 `s`로 써도 된다.
- **부분관측** 환경에서는 `obs`를 `s`로 쓰면 마르코프 성질이 깨져 **학습이 수렴하지 않거나, 수렴해도 잘못된 값**으로 수렴한다.
- “왜 내 에이전트가 수렴하지 않는가”를 debug할 때 **가장 먼저** 확인할 것: 현재 `obs`가 마르코프 성질을 만족하는가?

상태(state)

- 에이전트가 **알아야 하고, 알고리즘이 실제로 사용하는** 완전한 정보.
- 마르코프 성질이 **성립해야** 한다.

자주 하는 실수 (2/2): P 를 “한 번의 샘플”로 혼동

두 번째 흔한 실수: P 를 “한 번의 샘플”로 혼동.

- $P(s'|s, a)$ 는 확률 **분포**이지 한 번의 결과가 아니다.
- (0에서 right)을 100번 반복하면 **70번 1로, 30번 0으로** 갈 것이지만, 100번 중 한 번의 샘플이 “1로 갔다”는 사실은 $P = 0.7$ 을 **증명하지 않는다** — 추정치일 뿐이다.
- **모델 기반** 알고리즘(Ch. 4-5)은 이 추정치의 정확도에 **민감**하므로, 샘플 수와 추정 오차의 관계를 Chapter 5에서 자세히 다룬다.

요약

“관측 $\neq$ 상태”(역할의 차이), “ $P \neq$ 한 번의 샘플”(분포 vs 단일 결과). 두 실수는 모두 **확률적 MDP의 표기**와 **부분관측**을 혼동하는 것에서 온다.

실습: Gymnasium에서 $\mathcal{S}$, $\mathcal{A}$ 확인

이론의 다섯 요소를 실제 환경의 API와 대응시킨다. FrozenLake-v1(얼음 위를 미끄러지지 않게 건너는 문제)과 CartPole-v1 을 만들고, 각각의 $\mathcal{S}$ 와 $\mathcal{A}$ 를 출력:

```
import gymnasium as gym

# --- FrozenLake: 이산상태행동 · 공간 ---
fl = gym.make("FrozenLake-v1", is_slippery=True)
obs, info = fl.reset(seed=0)
print("FrozenLake S:", fl.observation_space)    # Discrete(16)
print("FrozenLake A:", fl.action_space)        # Discrete(4)
print("초기 상태:", obs, "(4x4 격자칸 16 중하나)")
fl.close()

print()

# --- CartPole: 연속상태 , 이산행동 ---
cp = gym.make("CartPole-v1")
obs, info = cp.reset(seed=0)
print("CartPole S:", cp.observation_space)      # Box(4,)
print("CartPole A:", cp.action_space)          # Discrete(2)
print("초기 상태:", obs, "[x, x_dot, theta, theta_dot]")
cp.close()
```

실습: 마르코프 성질을 샘플링으로 직접 검증

마르코프 성질이 “성립한다”는 말이 **검증 가능한 수학적 주장**인지 직접 확인. 간단한 **결정적** 환경(2-경로 환경: 상태 0에서 행동 0이면 1로, 행동 1이면 2로 전이)을 만들고, **같은 상태 1에 다른 경로**($0 \rightarrow 1$ vs $0 \rightarrow 2 \rightarrow 1$)로 도달한 뒤, 다음 상태 분포가 같은지 샘플링으로 확인:

```
import numpy as np

class TwoPathsEnv:
    """0 -> {1, 2} -> 1 -> 1(self-loop) 결정적 MDP."""
    def __init__(self, seed=0):
        self.rng = np.random.default_rng(seed)
        self.s = 0
    def reset(self, seed=None):
        if seed is not None:
            self.rng = np.random.default_rng(seed)
        self.s = 0
        return self.s
    def step(self, a):
        if self.s == 0:
            self.s = 1 if a == 0 else 2
        elif self.s == 1:
            self.s = 1 # self-loop, 경로무관
        else:
            self.s = 1 if a == 0 else 2
```

마르코프 검증: 핵심 패턴과 오차 분석

핵심 패턴: “같은 상태, 다른 경로, 다음 분포 비교”

이 패턴이 마르코프 성질 검증의 표준 절차다. 실습 노트북에서 이산/연속 환경 모두에서 더 자세히 한다.

- **결정적** 환경에서는 차가 **정확히 0**.
- **확률적** 환경에서는 샘플링 오차 ($\sim 1/\sqrt{N}$)가 있다.
- $N = 10000$ 이면 오차는 약 **0.003** 정도로, 0.01 임계값 안에 들어온다.

“마르코프 성질 성립”은 “같은 상태 s_t 에 서로 다른 경로로 도달했을 때, 다음 상태 분포가 같다”는 **검증 가능한** 수학적 주장이다. 결정적 환경에서는 차이 = 0, 확률적 환경에서는 $\sim 1/\sqrt{N}$ 샘플링 오차 안에서 0에 수렴한다.

실습: FrozenLake에서 P 를 실제로 추정하기

Gymnasium 환경의 P 는 내부 구현으로 감춰져 있지만, **샘플링으로 추정**할 수 있다:

$$P(s'|s, a) \approx \frac{N(s, a, s')}{N(s, a)}$$

((s, a) 에서 s' 로 전이한 횟수 / (s, a) 총 시도 횟수)로, 무작위 행동을 많이 하면 경험적 전이확률에 수렴.

```
import gymnasium as gym
import numpy as np

env = gym.make("FrozenLake-v1", is_slippery=True)
env.action_space.seed(0)
n_states = env.observation_space.n # 16
n_actions = env.action_space.n # 4
# count[s][a][s'] = (s,a)->s' 전이횟수
count = np.zeros((n_states, n_actions, n_states), dtype=int)

for ep in range(2000):
    s, _ = env.reset(seed=ep)
    done = False
    while not done:
        a = env.action_space.sample()
        s2, r, term, trunc, _ = env.step(a)
```

P 추정 결과: 모델 기반 vs 모델 없는

2000에피소드의 무작위 탐색으로 추정된 P 는, 실제 FrozenLake 구현(is_slippery=True일 때 intended 방향 2/3, 좌/우 1/6씩)에 **수렴**.

핵심: “ P 의 사용 방식”이 알고리즘의 가족을 나눈다

이 P_{est} 를 Chapter 4의 동적계획법에 그대로 대입하면 **모델 기반** 알고리즘을 돌릴 수 있고, P 를 무시하고 **경험만** 쓰면 **모델 없는** 알고리즘(Ch. 6-)을 돌릴 수 있다.

같은 환경, 같은 샘플, 다른 “ P 의 사용 방식”이 알고리즘의 가족을 나눈다.

모델 기반(Ch. 4-5): 추정된 P 를 명시적으로 써서 계획. 모델 없는(Ch. 6-11): P 를 전혀 쓰지 않고 경험(실제 전이 샘플)에서만 가치를 배운다. 이 “첫 갈림길”이 이 학기의 알고리즘 지도를 결정한다.

이 절을 마무리하며: 강화학습의 “문법”

이 절에서 만든 것을 한 줄로 요약:

$$\text{강화학습 문제} = (\mathcal{S}, \mathcal{A}, P, R, \gamma) + \text{마르코프 성질}$$

이 형식이 성립하면:

- ① 가치를 $V^\pi(s)$ 와 같은 “**상태에만 인자화된 함수**”로 정의할 수 있고(3.3),
- ② **벨만방정식**이라는 한 스텝 재귀식으로 가치를 계산할 수 있고(3.3),
- ③ 정책평가 · Q-learning · DQN · PPO 까지 — **이 형식 위에서 쓰인 모든 알고리즘**을 도출할 수 있다.

마르코프 성질이 “깨진” 문제(POMDP)

이 “문법”의 **전제가 깨진** 것이므로, “상태를 재설계해서 성질을 다시 성립시키는 것”이 **항상 첫 번째 해결책**.

FAQ (1/2)

● Q. 마르코프 성질이 실제 문제에서 항상 성립하나요?

- ▶ 엄밀하게는 거의 항상 근사.
- ▶ 예: 자율주행에서 “카메라 한 장”만 상태로 쓰면 **속도·가속도 정보가 빠져** 마르코프 성질이 깨진다 (다음 상태를 예측하려면 “지금 얼마나 빠르게 움직이고 있었는가”가 필요한데, 정지 이미지 한 장에는 그 정보가 없다).
- ▶ 실전: **최근 몇 프레임을 함께 묶어** “상태”로 재정의하거나, **신경망이 과거 정보를 요약한 은닉 상태**를 상태의 일부로 쓰는 방식 (Ch. 11 RNN 은닉 상태와 같은 아이디어)으로 다시 성립.

● Q. 밴딧(Ch. 2)도 MDP의 특수한 경우인가요?

- ▶ **그렇다** — $\mathcal{S} = \{s_0\}$ (상태 하나)에서 어떤 행동을 해도 **항상 같은 상태로 “전이”**하는 특수한 MDP.
- ▶ Ch. 2.3에서 밴딧과 MDP를 “별도”로 소개했지만, 이렇게 보면 이번 장의 **모든 정의**(가치함수, 벨만방정식)가 밴딧에도 **퇴화된 형태로** 그대로 적용.

FAQ (2/2)

● Q. 보상을 10배 키우면 최적 정책이 바뀌나요?

- ▶ 바뀌지 않는다 — 모든 $Q^\pi(s, a)$ 가 10배가 되지만, $\arg \max_a Q^\pi(s, a)$ 는 **변하지 않음**.
- ▶ 그러나: 보상의 상수 offset(모든 보상에 같은 상수 c 를 더하기)도 최적 정책을 바꾸지 않음(모든 Q 에 $c/(1 - \gamma)$ 가 균일하게 더해져 argmax 보존).
- ▶ 반면 상태/행동별로 불균일하게 보상을 조정하면(예: 한 상태에만 벌금 추가) 정책이 **바뀔 수** 있다.
- ▶ 보상의 “스케일”은 안전, “형태”는 위험하다.

● Q. γ 가 MDP의 “다섯 요소” 중 하나인데, 왜 3.2에서 자세히 다루나요?

- ▶ γ 의 수학적 역할(무한급수 수렴 보장, 가치함수 유계 보장)은 이번 절의 정의에서 이미 고정되어 있다 — $\gamma \in [0, 1)$ 으로 써둔 것이 그 전부.
- ▶ γ 의 실무적 의미(할인율, “지금 vs 미래” 교환 비율, 0.9~0.99로 고르는 기준)은 3.2에서 누적 보상 G_t 의 정의를 세울 때 자연스럽게 나온다.

FAQ (3/3)

● Q. 터미널 상태(**terminal state**)는 MDP의 다섯 요소에서 어디에 들어가는가?

- ▶ 터미널 상태는 $\mathcal{S}$ 의 원소지만, **전**이 행동이 제한된 특수한 상태 — 보통 **self-loop**(자기 자신으로 확률 1 전이)로 표현하고, **보상 0**을 준다.
- ▶ “종료”의 의미를 MDP 형식 안에 넣는 **표준 방식**이 바로 이것.
- ▶ Ch. 4 정책평가 코드에서 볼 수 있듯, 터미널 상태를 self-loop으로 표현하면 벨만방정식이 터미널에서도 **같은 형태**($V(s_{\text{term}}) = 0$)를 유지해 **코드 분기가 필요 없다**.

● Q. **Gymnasium**의 **terminated** 와 **truncated** 는 MDP의 어떤 것에 해당하는가?

- ▶ **terminated** = **진짜 종료**(터미널 상태 도달, 게임 끝) → MDP의 **터미널 상태**와 정확히 대응.
- ▶ **truncated** = **시간 한도 도달**(게임은 계속됐을 것이지만 학습 편의상 끊음) → MDP 형식에는 **직접 대응하지 않는 학습 장치**.
- ▶ 가치함수 업데이트 시 **terminated** 는 “**미래 가치 0**”로, **truncated** 는 “**다음 상태의 가치를 그대로**”로 처리해야 한다는 차이(1.3절에서 봤듯)가 여기에서 온다.

3.2 누적 보상, 할인율, 그리고 실습

이 절의 흐름

3.1절에서 MDP의 다섯 요소 ($\mathcal{S}, \mathcal{A}, P, R, \gamma$) 를 “나열”하기만 했다. 이 절은 그중 R (보상)과 γ (할인율)에 집중해서, 강화학습이 실제로 **최대화 하는 대상 — 리턴 — 을 손으로 코드로 계산할 수 있게 만든다.**

수업의 흐름:

- ① **리턴**(할인된 누적 보상)의 정의와 유한/무한 경우의 계산법.
- ② “할인율 0.95”가 실제로는 “몇 스텝짜리 시야”를 뜻하는지 — **유효 시계(effective horizon)**로 바꾸는 법.
- ③ γ 를 바꾸면 “같은 환경에서 다른 행동을 **최적으로**” 만드는 예 — 할인율이 단순 정규화 장치가 아니라 **목표를 바꾸는** 매개변수임을 확인.
- ④ Gymnasium에서 CartPole로 할인된 리턴을 실제로 누적.

리턴(return)의 정의

강화학습의 목표는 한 스텝의 보상이 아니라, 앞으로 받을 **보상들의 합** — **리턴(return)** G_t — 을 최대화하는 것이다:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$$

왜 할인율 γ 가 필요한가

- ① **수학적으로:** $\gamma < 1$ 이면 보상이 유계(bounded)인 한 이 **무한급수가 항상 수렴**(기하급수).
 $\gamma = 1$ 이면 끝나지 않는 과업에서 리턴이 **무한대로 발산**할 수 있다.
 - ② **직관적으로:** “지금 당장의 보상 1”이 “10스텝 뒤의 보상 1”보다 더 가치 있다고 보는 경우가 많다(사람의 시간 선호, 금융의 이자율과 같은 방향).
- γ 가 0에 가까우면 **근시안적**(당장의 보상만 중시), 1에 가까우면 **장기적**(먼 미래까지 충분히 고려).

닫힌 형태로 잘 풀리는 무한합

이 무한합은 실제로 **닫힌 형태로** 잘 풀린다. $\gamma < 1$ 이면 기하급수의 합 공식을 그대로 쓸 수 있다.

보상이 매 스텝 같은 값 R 으로 고정된 경우:

$$G_t = R + \gamma R + \gamma^2 R + \cdots = \frac{R}{1 - \gamma}$$

- $\gamma = 0.9, R = 1$ 이면 $G_t = \frac{1}{0.1} = 10$ — “매 스텝 1을 무한히 받는 환경”의 리턴이 10이라는 뜻.
- γ 가 1에 다가갈수록 이 값은 $\frac{1}{1-\gamma}$ 처럼 **무한대로 올라간다** — “발산”의 구체적 모양.

보상이 매 스텝마다 다른 일반적 경우: 한 에피소드가 유한한 스텝 T 뒤에 끝나면(터미널 도달 뒤 보상은 모두 0) **무한합은 유한합으로 잘린다:**

$$G_0 = \sum_{k=0}^T \gamma^k R_k = R_0 + \gamma R_1 + \gamma^2 R_2 + \cdots + \gamma^T R_T$$

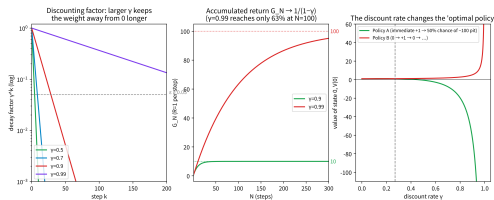
“무한합”이라는 **모양만** 무서울 뿐, 실제로는 **마지막 보상 R_T 의 계수 γ^T 가 아주 작으므로** 유효한 항은 앞에 몇 개뿐이다 — “몇 개쯤이 유효한지”를 숫자로 잡는 것이 바로 다음 절의 **유효 시계다**.

매 스텝 보상 $R = 1$ 일 때: 유한합이 닫힌 형태로 접근

매 스텝 보상 $R = 1$ 일 때, 유한합 $G_N = \sum_{k=0}^{N-1} \gamma^k$ 이 닫힌 형태 $\frac{1}{1-\gamma}$ 로 어떻게 다가가는지:

N	$G_N (\gamma = 0.9)$	$G_N (\gamma = 0.99)$
5	4.10	4.90
10	6.51	9.56
20	8.78	18.21
50	9.95	39.50
100	10.00	63.40
∞	10	100

- $\gamma = 0.9$ 는 20스텝이면 이미 닫힌 형태의 **87.8%**, 100스텝이면 99.997%.
- $\gamma = 0.99$ 는 100스텝이 돼도 **63.4%**에 불과.



(가운데) 매 스텝 보상 1의 누적 리턴이 $1/(1-\gamma)$ 로 수렴.

결론

γ 가 클수록 합산해야 하는 항의 수가 늘어 “더 장기적”이라는 말의 구체적 수학적 의미.

유효 시계: $\gamma = 0.95$ 는 몇 스텝짜리 시야인가

할인율을 “몇 %씩 깎는 비율”로만 이해하면, “0.95와 0.9의 차이가 실제로 크다”를 느끼기 어렵다.

재정의

“할인 계수 γ^k 가 ϵ 아래로 떨어지는 첫 스텝 k 를 H 라 하자” — 이 H 를 **유효 시계 (effective horizon)**라고 부른다. “에이전트는 대략 H 스텝 까지 미래를 볼 수 있다”고 읽을 수 있다.

$\gamma^H = \epsilon$ 양변에 로그를 취하면:

$$H = \frac{\log \epsilon}{\log \gamma}$$

$\epsilon = 0.05$ 로 잡으면(= 5% 이하로 깎인 보상부터 “무시”):

γ	유효 시계 H ($\epsilon = 0.05$)	해석
0.5	4.3 → 약 4스텝	5스텝 뒤 보상은 이미 3%
0.9	28.4 → 약 28스텝	약 28스텝 뒤 보상은 5% 이하
0.95	58.4 → 약 58스텝	CartPole 에피소드(500)의 $\sim 1/9$
0.99	298.1 → 약 298스텝	500스텝 에피소드의 60%까지 보임

유효 시계: 반대로 쓰기 + “5%”의 의미

반대로 쓰는 법(실전 팁)

유효 시계 공식을 뒤집으면 $\gamma = \epsilon^{1/H}$ 다. “에이전트가 H 스텝을 볼 수 있게 하라”고 정하면 γ 가 **자동으로** 결정된다:

- 500스텝짜리 에피소드를 다 보려면 $\gamma = 0.05^{1/500} \approx 0.994$.
- 100스텝짜리 환경이면 $\gamma = 0.05^{1/100} \approx 0.97$.

“**환경의 수명에 γ 를 맞춘다**”는 실전 관행이 바로 이 식에서 온다.

“5%로 깎였다”가 왜 “사실상 무시”인가?

H 스텝 **이후**에 들어오는 **모든** 보상의 합은, 그 직전 스텝의 보상에 $\frac{\epsilon}{1-\gamma}$ 를 곱한 것 **이상으로 클 수 없다**. $\gamma = 0.9, \epsilon = 0.05$ 이면 28스텝 이후의 **모든** 미래 보상의 합이 28스텝째 보상의 $0.05/0.1 = 0.5$ 배를 넘지 못한다.

“무한히 먼 미래”가 실제로는 **한 스텝짜리 상한**에 압축되는 것이다.

할인율은 “목표”를 바꾼다: 예 1

“할인율은 발산을 막기 위한 수학 장치”라는 설명은 **반쪽**이다. **유한한 에피소드에서는 $\gamma = 1$ 이어도 발산하지 않는다** — 그런데도 $\gamma < 1$ 을 쓰는 이유는, γ 가 어떤 정책을 “최적”으로 만드는지를 실제로 바꿀 수 있기 때문이다.

예 1 (당장 1 vs 5스텝 뒤 3): 매 스텝 보상 $R_0 = 1, R_1 = R_2 = R_3 = 0, R_4 = 3$ 인 환경.

γ	G_0 계산	G_0
1	$1 + 3$	4.00
0.9	$1 + 0.9^4 \times 3$	2.9683
0.5	$1 + 0.5^4 \times 3$	1.1875

- $\gamma = 0.5$ 로 하면 5스텝 뒤의 3은 **0.19로 사실상 사라졌다.**
- 같은 환경인데, γ 만 달리면 “5스텝 뒤의 3을 위해 지금을 견디는 것”의 가치가 1.97에서 0.19로 줄어든다.
- 어느 시점($3\gamma^4 = 1$, 즉 $\gamma \approx 0.76$) 이후에는 “당장의 1만 챙기는 행동”이 “5스텝을 버텨서 3을 받는 행동”보다 리턴이 커진다.

같은 환경에서, γ 만 바뀌도 최선의 행동이 바뀌는 것이다.

할인율은 “목표”를 바꾼다: 예 2 (3상태 MDP)

예 2: 상태 0, 1, 2에, 각 상태에서 두 행동 A·B.

현재 상태	행동 A	행동 B
0	$\rightarrow 1$, 보상 +1	$\rightarrow 1$, 보상 +1
1	$\rightarrow 2(50\%), 0(50\%), +2$	$\rightarrow 0(\text{확정}), 0$
2 (루프)	자기 자신, -10	자기 자신, -10

- **정책 A**(항상 A): 상태 1에서 A를 하면 50% 확률로 상태 2로 넘어가, 그 뒤 **매 스텝 -10 을 영구히** 받는다(상태 2는 “게임 종료”가 아니라 “-10 을 계속 주는 루프”).
- **정책 B**(항상 B): 상태 1에서 보상을 0으로 받고 상태 0으로 돌아가 $0 \xrightarrow{+1} 1 \xrightarrow{0} 0 \rightarrow \dots$ 의 **안전하지만 보상이 낮은 루프**.

터미널이 아니라 -10 루프로 모델링한 이유: 터미널이었다면 $V(2) = 0$ 이 되어 “구덩이가 γ 에 의해 얼마나 할인되는지”를 볼 수 없기 때문이다.

예 2: γ 를 올리면 답이 뒤집힌다

$\gamma = 0.9$ 로 두 정책의 가치를 반복 대입(3.3절 벨만방정식 고정점)으로 풀어보면:

$$V^A = [-63.36, -71.51, -100], \quad V^B(0) = \frac{1}{1 - \gamma^2} = \frac{1}{0.19} \approx 5.26$$

- 벨만방정식 세 등호: $V(0) = 1 + 0.9 V(1)$, $V(1) = 2 + 0.9 (0.5 V(2) + 0.5 V(0))$, $V(2) = -10$ (코드로 계산: $V^A(0) = -63.3613$).
- $V^A(2) = -10/(1 - 0.9) = -100$ — “한 번 -10 루프에 빠지면 그 상태의 가치는 -100 ”.
- $\gamma = 0.9$ 에서는 **정책 A(-63.36)가 정책 B(5.26)보다 훨씬 나쁘다** — “50% 확률로 -100 짜리 구덩이”가 2스텝짜리 $+2$ 보상의 기대를 훨씬 넘는다.

그런데 γ 를 낮추면 답이 뒤집힌다: $\gamma = 0.1$ 이면 $V^A(0) = 1.15 > V^B(0) = 1.01$ — **정책 A가 오히려 좋다**. $\gamma \rightarrow 0$ 극한에서는 “앞으로 1스텝만” 보므로 상태 1에서 $+2$ 를 즉시 주는 A가 불리하지 않다.

그림: 오른쪽 패널에서 두 정책의 상태 0 가치 곡선이 크로스오버 $\gamma^* \approx 0.27$ 에서 교차.

크로스오버 지점: “할인율은 목표를 바꾼다”의 의미

γ 를 0에서 1 사이로 계속 올리며 계산하면, $\gamma \approx 0.27$ 라는 “크로스오버” 지점에서 두 정책의 상태 0 가치가 같아지고, 그 이상($\gamma \gtrsim 0.27$)에서는 정책 B가 일관되게 좋아진다.

핵심

같은 환경, 같은 두 정책인데 γ 만 바꾸면 “어느 정책이 옳은가”의 답이 뒤집힌다 ($\gamma \lesssim 0.27$ 에서 A, $\gamma \gtrsim 0.27$ 에서 B).

이것이 “할인율은 목표를 바꾼다”의 구체적 의미. γ 는 단순 정규화 상수가 아니라, 어떤 행동을 “옳은” 행동으로 만드느냐를 정하는 설계 변수.

역사적 맥락

“현재 보상을 미래 보상의 기댓값에 더해 재귀적으로 정의한다”는 관점은 1950년대 벨만(Bellman, 1957)의 동적계획법에서 나왔다. “복잡한 문제를 한 번에 풀지 말고, 한 스텝만 보고 나머지는 부분 문제로 위임하라”는 통찰이 바로 3.3절의 벨만방정식인데, 그 식의 “한 스텝”을 지금과 미래로 나누어 가중하는 비율이 바로 γ 다. 할인율은 “수학의 편의”가 아니라, 결정 이론의 시간 가중(time discounting)을 MDP에 이식한 것이다.

실습: CartPole에서 할인된 리턴 직접 누적

CartPole은 매 스텝 살아 있으면 +1 을 주므로(1.3절), **보상이 전부 1**인 특수한 경우 — 리턴은 “얼마나 오래 버텼는가”의 **할인된 버전**.

```
import gymnasium as gym

env = gym.make("CartPole-v1")
obs, info = env.reset(seed=0)

# 5개의 MDP 요소를 실제로 확인
print("상태 공간 S:", env.observation_space)  # 4차원 연속 벡터
print("행동 공간 A:", env.action_space)        # 개 2 이산 좌우 (/)

total_return, gamma, discount = 0.0, 0.95, 1.0
for _ in range(10):
    action = env.action_space.sample()
    obs, reward, terminated, truncated, info = env.step(action)
    total_return += discount * reward  # r_{t+1} 직접 누적
    discount *= gamma
    if terminated or truncated:
        break
print("10 스텝 동안의 할인된 리턴 G_0:", round(total_return, 3))
env.close()
```

CartPole 10스텝: $\gamma = 0.95$ 의 할인 효과

$\gamma = 0.95$, 시드 0이면 10스텝 모두 살아서 매 스텝 +1 을 받으므로:

$$G_0 = 1 + 0.95 + 0.95^2 + \dots + 0.95^9 = \frac{1 - 0.95^{10}}{1 - 0.95} \approx 8.02$$

실제로 출력되는 **8.025**가 이 값.

- 10스텝째 보상의 계수 $0.95^9 \approx 0.63$ 이므로, “10스텝”을 다 더한 10보다 **왜 작다**.
- 이미 10스텝 만에 “할인의 효과”가 눈에 보인다.

베이스라인 잡기

무작위 정책으로 200개 시드를 돌려, γ 의 크기가 “같은 환경”의 베이스라인 리턴을 어떻게 **바꾸는지**를 확인.

다음 프레임에서 코드와 출력. 핵심: “버틴 스텝 수”는 γ 와 무관한데, “리턴의 숫자”는 γ 에 따라 크게 바뀐다.

베이스라인: γ 가 리턴을 어떻게 바꾸나

```
import gymnasium as gym
import numpy as np

env = gym.make("CartPole-v1")
for gamma in [0.9, 0.95, 0.99]:
    returns = []
    for seed in range(200):
        s, _ = env.reset(seed=seed)
        env.action_space.seed(seed)
        total, discount = 0.0, 1.0
        while True:
            a = env.action_space.sample()
            s, r, term, trunc, _ = env.step(a)
            total += discount * r
            discount *= gamma
            if term or trunc:
                break
        returns.append(total)
    returns = np.array(returns)
    print(f"gamma={gamma}: 무작위평균리턴 = {returns.mean():.2f} "
          f"스텝(500 최대: {(1-gamma**500)/(1-gamma):.2f})")
env.close()
```

베이스라인 해석: 같은 γ 안에서만 비교

- 버틴 스텝 수는 γ 와 무관하게 평균 24.1스텝 (1.3절의 베이스라인).
- 리턴의 숫자는 γ 에 따라 8.62 $\rightarrow$ 13.07 $\rightarrow$ 20.77로 크게 바뀐다.
- 이는 “할인율이 보상의 총량이 아니라, **언제의 보상을 얼마나 헤아리는가**를 정한다”는 뜻.

주의: 서로 다른 “화폐 단위”

$\gamma = 0.9$ 의 “평균 8.62”와 $\gamma = 0.99$ 의 “평균 20.77”을 같은 스케일로 비교하면 안 된다 — 서로 다른 **화폐 단위(할인 기준)**의 숫자.

같은 γ 안에서만 리턴의 크기를 비교하는 것이 정석.

이 “화이트라벨” 같은 주의는 Ch. 5-7에서 **리플레이 버퍼**에 여러 환경.다른 γ 로 모은 전이를 함께 쓸 때 실제로 문제가 된다.

그림(오른쪽): 매 스텝 보상 1의 10스텝 에피소드에서 “할인 없이 더함”은 γ 와 무관하게 10, “ $\gamma = 0.95$ 의 정확한 할인”은 8.03, “두 번 할인”은 6.58 — 두 번 할인은 사실상 γ^2 를 할인율로 쓰는 것과 같으므로 곡선이 가장 빨리 0으로 곤두박질한다.

자주 하는 실수: 할인율을 빼먹기 / 두 번 할인

실수 1: 할인율을 빼먹고 그냥 보상을 다 더하기

- 매 스텝 보상이 1씩 5번($R_0 = \dots = R_4 = 1$) 들어오면:

$$G_0 = 1 + 0.9 + 0.81 + 0.729 + 0.6561 = 4.0951$$

단순 합(5)보다 **항상 작다**(할인율이 “미래 보상을 깎아서” 계산).

- `total_return += reward`처럼 `discount`를 곱하는 걸 빼먹으면: (1) 값 자체가 G_t 의 정의와 달라지고, (2) $\gamma = 1$ 인 것과 같아져 **끝나지 않는 과업에서 발산** 위험.
- 디버깅: “리턴이 이상하게 크다”면 **먼저 할인율 곱셈이 빠졌는지** 확인.

실수 2: 할인을 두 번 적용하기(실수 1의 정반대)

- `total_return += discount * reward`를 쓰면서 **동시에** `reward = reward * gamma`로 보상을 먼저 깎아놓으면, **두 번** 깎인 γ^{2k} 가 적용.
- $\gamma = 0.95$ 이면 10스텝째 보상의 계수가 $0.95^{10} \approx 0.60$ 이 아니라 $0.95^{20} \approx 0.36$ 으로, **두 배 이상** 빨리 0에 가까워진다.
- “에이전트가 이상하게 **근시안적**으로 느껴지는가?”가 의문이면 할인 곱셈이 두 군데에 있는지 코드 전체를 grep. (위 CartPole 예에서 10스텝 리턴이 8.025 대신 6.58이면 “두 번 할인”의 전형적 신호.)

확인 문제 (3.2)

- ① $\gamma = 0.9, \epsilon = 0.05$ 일 때 유효 시계 H 를 $H < \log(0.05)/\log(0.9)$ 로 직접 계산. 로그를 쓰지 않고 $0.9^{28}, 0.9^{29}$ 를 직접 곱해서도 검증. 이 $H = 28$ 이 “에이전트는 28스텝 이후의 보상을 5% 이하로 취급한다”는 뜻과 일치하는지 확인.
- ② 매 스텝 보상 $R = 1$ 인 환경에서, $\gamma = 0.9$ 와 $\gamma = 0.99$ 일 때 각각 “한 에피소드에서 받을 수 있는 **최대 리턴**”($1/(1 - \gamma)$)을 계산. 이 차이가 “ $\gamma = 0.99$ 를 쓰면 $\gamma = 0.9$ 때보다 **상대적으로** 더 먼 미래의 보상이 중요하다”는 뜻인지 한 문장으로 설명.
- ③ (개념) “할인율 0.95는 “5% 이자율””이라는 비유. 금융에서 “1년 뒤 1원”의 현재가치 $1/(1 + r)$ 과 비교해, 강화학습의 γ 가 “이자율”과 **같은 방향**인지, **다르다**는 점(예: $\gamma = 1$ 이 “이자율 0”이 아니라 “미래 보상을 무한히 중시”하는 극단)을 각 한 문장으로.

FAQ (1/2): γ 설정

● Q. γ 는 보통 얼마로 설정하나요?

- ▶ 0.9~0.99 사이가 흔.
- ▶ CartPole처럼 **에피소드가 몇백 스텝 이내**로 짧은 문제는 0.95~0.99, **아주 긴 호홉** (수천 스텝 이상)은 0.99~0.999처럼 1에 더 가까운 값을 써서 먼 미래의 보상도 무시되지 않게.
- ▶ 실전 하이퍼파라미터 튜닝에서는 γ 를 **학습률만큼이나** 중요한 변수로 취급 — 같은 알고리즘을 같은 환경에 돌려도 $\gamma = 0.9$ 와 $\gamma = 0.99$ 의 **최종 정책**이 다를 수 있기 때문.

● Q. 유한 에피소드 문제에서도 할인율이 꼭 필요한가요?

- ▶ 유한한 에피소드라면 $\gamma = 1$ 이어도 리턴이 **발산하지는 않는다**(합이 유한 개 항).
- ▶ 다만 $\gamma < 1$ 을 쓰면 “당장의 보상을 더 중시한다”는 **실용적 효과** + 학습 알고리즘의 **분산을 줄여 안정성을 높이는** 부수 효과.

FAQ (2/2): γ 의 극단

● Q. $\gamma = 0$ 는 무슨 뜻인가요?

- ▶ “당장의 스텝 **하나**의 보상뿐”을 본다는 뜻. $G_t = R_t$ 로 리턴이 즉시 보상과 같아지고, **근시안의 극단**.
- ▶ MDP는 퇴화하여 **다중팔 밴딧**(각 상태가 별개의 밴딧)이 된다.
- ▶ 드물지만 “보상이 이미 **누적된 값**으로 주어지는 환경”에서는 $\gamma = 0$ 으로 “최종 총점”을 한 번만 세는 것이 **이중 계산을 막는 정석**.

● Q. γ 를 0.999처럼 1에 아주 가깝게 하면 “더 좋은” 것 아닌가?

- ▶ (1) **수학**: $\gamma = 1$ 이면 끝나지 않는 과업에서 발산할 수 있다. $\gamma < 1$ 은 수렴을 수학적으로 보장.
- ▶ (2) **실용**: 1에 가까울수록 리턴의 **분산**이 커지고, 수렴이 **느려져** 학습이 불안정해질 수 있다.
- ▶ “1에 가까울수록 좋은가”는 **환경의 수명**과 **알고리즘의 분산**을 함께 저울질해야 정해지는 **설계 판단**.

FAQ (3/3): “할인 두 번”과 오프바이원

- Q. “할인된 리턴”을 코드로 계산할 때, $\text{discount} *= \text{gamma}$ 를 $\text{step}()$ 이후에 해야 하는 이유는?
 - ▶ $G_0 = R_0 + \gamma R_1 + \gamma^2 R_2 + \dots$ 의 정의에서, R_k 의 계수는 γ^k .
 - ▶ $\text{step}()$ 직후에 받는 reward는 이번 스텝의 R_t 이므로, 그 당시의 $\text{discount}(= \gamma^t)$ 를 곱한 뒤, 다음 스텝을 위해 $\text{discount} *= \text{gamma}$ 를 해야 한다.
 - ▶ 순서를 바꾸면 R_t 에 γ^{t+1} 이 걸려 모든 보상이 한 스텝 더 할인 — “할인을 두 번” 실수와는 다른, **오프바이원(off-by-one)** 버그.

3.2절 한 줄 요약

리턴 G_t 는 “한 스텝의 보상이 아니라, 앞으로 받을 보상의 **할인된 합**”이고, 할인율 γ 는 그 합을 수렴시키는 수학 장치이면서 동시에, 어떤 보상을 중시하는지를 정하는 **목표 변수**다.

같은 환경에서도 γ 를 바꾸면 “최적인 행동”이 바뀔 수 있다(예 2)는 점이, 할인율을 “단순 정규화 상수”가 아니라 **설계 결정**으로 봐야 하는 이유다.

3.3 가치함수와 벨만 기대방정식

이 절에서 어디로 가는지

지금까지 (3.1) MDP의 다섯 요소와 (3.2) 리턴 G_t ·할인율 γ 를 배웠다. 아직 “이 상태가 얼마나 좋은가”를 **한 숫자로** 말해주는 도구는 없다. 이번 절은 그 도구를 만든다.

흐름은 정확히 이 네 가지:

- ➊ **가치함수** V^π, Q^π — “이 상태(상태+행동)에서 정책 π 를 따랐을 때 기대 리턴”을 정의.
- ➋ **벨만 기대방정식** — 그 기대 리턴을 “지금 보상 + 다음 상태 가치(할인)”라는 **한 스텝짜리 재귀식**으로 변환.
- ➌ **수치 계산** — 그 재귀식을 “값이 안 바뀔 때까지 반복 대입”하는 절차(반복적 정책평가)로 풀고, 손계산값과 검산.
- ➍ **연결고리** — 이 재귀식이 다음 장 동적계획법의 **정책평가**와, 궁극적으로는 Ch. 4의 **최적** 방정식(max 이 붙은 것)으로 이어지는 길목임을 보임.

이 네 가지를 다 마치면, “MDP의 한 상태의 가치를 재귀적으로 계산하는 절차”를 **손으로, 코드로, 그리고 수학적으로 유일하게 보장받는다**는 세 가지 언어로 말할 수 있다.

가치함수: 리턴의 기댓값

리턴 G_t 는 **무작위 변수**다(전이도 확률적일 수 있고, 정책도 확률적일 수 있다) — 그래서 “이 상태가 얼마나 좋은가”를 하나의 숫자로 요약하려면 **기댓값**을 쓴다.

정책(policy) π (각 상태에서 어떤 행동을 할지 정하는 규칙, 확률적일 수도 있어 $\pi(a|s)$ 로 쓴다)를 따를 때:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s] \quad (\text{상태가치함수, state-value})$$

“상태 s 에서 시작해서 π 를 계속 따른다면, 앞으로 받을 할인된 보상의 합이 **평균적으로** 얼마인가”.

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a] \quad (\text{행동가치함수, Q-함수})$$

“상태 s 에서 **행동** a 를 하고, 그 **이후**로는 π 를 따른다면”의 가치를 재는다.

표로 비교: V^π vs Q^π

	$V^\pi(s)$ 상태가치	$Q^\pi(s, a)$ 행동가치
고정하는 것	상태 s (행동은 π 가 정함)	상태 s 와 첫 행동 a
무작위성	행동 과 전이 모두	첫 행동만 제거, 전이만 남음
매기는 대상	S 에 하나씩	$S \times \mathcal{A}$ 에 하나씩
“무엇을” 측정	그 상태에서 π 를 따를 때의 기대 리턴	a 를 한 다음 π 를 따를 때의 기대 리턴
주로 쓰이는 곳	“이 상태가 좋은가” (정책평가, 가치 추정)	“이 상태에서 어떤 행동이 좋은가” (정책비교/향상, Q-학습)

두 개를 왜 둘 다 정의하는가

둘 다 “리턴의 기댓값”인데, **어디를 고정한지가 다르다**. 동적계획법의 정책향상(policy improvement)은 “지금 이 상태에서 **어떤 행동이 가장 좋은가**”를 물어보는데, 그 물음은 $V^\pi(s)$ 만으로는 답할 수 없고, 각 행동의 가치 $Q^\pi(s, a)$ 를 **행동마다 비교**해야 한다.

즉 Q 가 있으면 **비교**할 수 있고, V 만 있으면 **비교**할 수 없다. 그래서 둘 다 필요하다.

직관: “이 상태의 가격표”

- V^π 는 상태 공간에 **매 상태마다 하나의 숫자**를 매기는 함수 — 상태별 “**가격표**”다.
- Q^π 는 “상태-행동” **조합**마다 하나의 숫자를 매기는, 좀 더 **잘게 쪼갠** 가격표.
- “이 상태에서 앞으로 평균적으로 얼마를 벌 수 있나”라는 질문에 $V^\pi(s)$ 가 바로 답을 주는 숫자.

중요: “실제 리턴”이 아니라 “기댓값”

이 숫자는 “**실제로 벌어지는 리턴**”이 아니라 그 **기댓값**이다. 같은 상태에서 시작해도 전이가 확률적이면 매 에피소드마다 리턴이 다르다 — $V^\pi(s)$ 는 “이 상태에서 출발하면 **평균적으로** 이 정도”를 말해주는 **요약 통계**다.

벨만 기대방정식: 무한합을 재귀식으로

G_t 의 정의를 **한 스텝만** 풀어써보면:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots = R_t + \gamma(R_{t+1} + \gamma R_{t+2} + \cdots) = R_t + \gamma G_{t+1}$$

즉 리턴은 “**지금 당장의 보상 + 할인된 다음 시점 리턴**”으로 재귀적으로 쓸 수 있다. 이 관찰을 V^π 의 정의에 대입하면 (기댓값의 **선형성**을 이용):

$$V^\pi(s) = \mathbb{E}_\pi[R_t + \gamma G_{t+1} \mid s_t = s] = \mathbb{E}_\pi[R_t + \gamma V^\pi(s_{t+1}) \mid s_t = s]$$

이게 (Bellman expectation equation)

무한히 먼 미래까지 다 더해야 할 것 같던 $V^\pi(s)$ 가, “**지금 보상 + 다음 상태의 가치(할인됨)**”이라는 **한 스텝짜리 재귀식**으로 압축된다.

왜 G_{t+1} 을 $V^\pi(s_{t+1})$ 로 바꿀 수 있는가

이것이 이 절에서 가장 “왜?”가 필요한 순간.

- G_{t+1} 은 **구체적 경로** 하나(실제로 벌어진 특정 보상 열)의 리턴이고, $V^\pi(s_{t+1})$ 은 “상태 s_{t+1} 에서 출발하는 **모든 경로**의 리턴의 **평균**”이다.
- 이 둘이 같아도 되는 이유는 **마르코프 성질**(3.1절). 다음 상태 s_{t+1} 이 정해졌다면, 그 뒤에 이어질 보상 열의 분포는 **어떻게 s_{t+1} 에 왔는지에 의존하지 않는다**.
- 따라서 “여기까지 어떤 경로를 통해 왔느냐”는 정보 전체를 버리고 “지금 s_{t+1} 에 있다”는 사실 **하나**로만 조건부 지을 수 있다 — 그 조건부 기댓값이 바로 $V^\pi(s_{t+1})$ 의 정의.

요약

벨만방정식은 결국 **마르코프 성질을 “한 스텝만” 적용한 결과**인 셈이다. 3.1절의 성질이 왜 강화학습의 뼈대인가를, **수식 한 줄**이 정확히 보여준다.

이 재귀식이 얼마나 강력한가 — 상태가 100만 개여도

재귀식의 가장 큰 쓸모는 **계산량**이다.

- 정의대로 $V^\pi(s)$ 를 계산하려면 상태 s 에서 출발해 **무한히 많은 경로**를 모두 탐색하고 각각의 할인 리턴을 평균내야 한다 — 상태가 100만 개면 경로 수는 사실상 **무한대로 폭증**.
- 벨만방정식은 “상태 s 의 가치를 알려면 **인접 상태들** s' 의 가치만 알면 된다”고 말한다.
- 즉 100만 개의 상태를 모두 한 번씩 “인접 상태의 가치에 지금 보상을 더하는” 일로 계산 — **경로 탐색이 상태별 한 스텝 연산으로 압축**.

동적계획법의 핵심 아이디어

이 “무한 경로 $\rightarrow$ 인접 상태만”이 바로 **동적계획법 (dynamic programming)**이라는 이름의 **핵심 아이디어**이며, 다음 장의 **정책평가 알고리즘 전체**가 이 한 줄의 직접적 결과물이다.

그림: 벨만방정식의 첫 스텝 분해 — 확률적 정책 (각 1/2)으로 두 가지가 갈렸다가, 같은 상태 $s_1 \cdot s_0$ 로 다시 모인다.
“같은 상태는 같은 가치”를 하나의 노드로 병합하는 것이 무한 경로를 상태 수(여기서 2개)로 줄이는 재귀식의 기하학적 모습.

$G_t = R_t + \gamma G_{t+1}$ 의 분화 — 같은 상태는 같은 가치

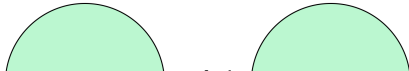


그림 읽어보기: “같은 상태는 같은 가지”

- 왼쪽 노드는 “지금” 상태 s_0 이다. s_0 에서 두 행동 a_0 (보상 1), a_1 (보상 3)이 각 **1/2 확률**로 선택되고, 둘 다 상태 s_1 으로 전이(각각 $p = 1$).
- 다시 s_1 에서 한 행동(보상 2)으로 s_0 으로 돌아온다.
- **주목할 점:** 세 번째 수준에서 s_0 가 두 번 등장한다 — 이 두 가지는 “다음 상태가 같으면 그 뒤가 같다”는 마르코프 성질에 의해 **같은 기댓값** $\gamma \cdot V(s_0)$ 를 공유한다.

핵심 작업

“같은 상태를 하나의 노드로 병합”하는 작업이 벨만방정식의 기하학적 모습이다. **경로 수(무한)를 상태 수 (여기서는 2개)로 줄이는** 것이다.

확률적 MDP로 손계산: MDP 정의

위 재귀식은 “이론”이지만, 실제로 V 와 Q 가 서로를 정의하는 “**동전의 양면**”인지 숫자로 확인. 이번에는 **각 상태에서 행동을 고르는 정책이 확률적인 MDP**. (3.2절의 예는 모든 전이가 **결정적**이었으므로.)

MDP 정의: 상태 2개(S_0, S_1), $\gamma = 0.9$.

- S_0 에서 정책 π 는 a_0 를 1/2, a_1 를 1/2 확률로 고른다.
 - ▶ a_0 : 즉시 보상 $R = 1$, **반드시** S_1 로 전이.
 - ▶ a_1 : 즉시 보상 $R = 3$, **반드시** S_1 로 전이.
- S_1 에서 정책 π 는 **항상** a_0 를 고른다:
 - ▶ a_0 : 즉시 보상 $R = 2$, **반드시** S_0 로 전이.

“확률적 정책 + 결정적 전이” 구조. 전이가 결정적이므로 무작위성은 **정책의 행동 선택**에만 있고, 이것이 V 와 Q 를 가중평균으로 연결한다.

손계산: Q 부터, V 는 가중평균

Q^π 부터 계산 — 각 (상태, 행동)의 행동가치는 “그 행동의 즉시 보상 + 다음 상태의 가치(할인)”:

$$Q^\pi(S_0, a_0) = 1 + 0.9 V^\pi(S_1), \quad Q^\pi(S_0, a_1) = 3 + 0.9 V^\pi(S_1)$$

$$Q^\pi(S_1, a_0) = 2 + 0.9 V^\pi(S_0)$$

V^π 는 정책의 행동을 확률로 가중평균:

$$V^\pi(S_0) = \frac{1}{2}Q^\pi(S_0, a_0) + \frac{1}{2}Q^\pi(S_0, a_1)$$

$$= \frac{1}{2}(1 + 0.9V^\pi(S_1)) + \frac{1}{2}(3 + 0.9V^\pi(S_1)) = \frac{1}{2} \cdot 4 + 0.9 V^\pi(S_1) = 2 + 0.9 V^\pi(S_1)$$

S_1 에서는 행동을 하나만(확률 1로 a_0) 고르므로:

$$V^\pi(S_1) = Q^\pi(S_1, a_0) = 2 + 0.9 V^\pi(S_0)$$

손계산: 연립방정식 풀기와 검산

연립방정식 풀기 — $V^\pi(S_1) = 2 + 0.9 V^\pi(S_0)$ 를 첫 번째 식에 대입:

$$V^\pi(S_0) = 2 + 0.9(2 + 0.9 V^\pi(S_0)) = 2 + 1.8 + 0.81 V^\pi(S_0) = 3.8 + 0.81 V^\pi(S_0)$$

$V^\pi(S_0)(1 - 0.81) = 3.8$ 이므로 $V^\pi(S_0) = 3.8/0.19 = \mathbf{20}$. 다시 대입하면

$$V^\pi(S_1) = 2 + 0.9 \cdot 20 = \mathbf{20}.$$

검산: Q 값으로 되돌아와서 $V = \sum_a \pi(a|s) Q$ 가 정말 성립하는지:

- $Q^\pi(S_0, a_0) = 1 + 0.9 \cdot 20 = 19$, $Q^\pi(S_0, a_1) = 3 + 0.9 \cdot 20 = 21$.
 $\frac{1}{2}(19) + \frac{1}{2}(21) = 20 = V^\pi(S_0) \quad \checkmark$
- $Q^\pi(S_1, a_0) = 2 + 0.9 \cdot 20 = 20 = V^\pi(S_1) \quad \checkmark$

둘 다 정확히 20. “ V 는 각 행동을 정책 확률로 가중평균한 Q 의 합”이 수식 수준에서 성립한다.

이 예가 보여준 것: 기대값으로 값이 결정된다

- (1) V 는 각 행동을 정책 확률로 **가중평균**한 Q 의 합과 **정확히 일치** — “동전의 양면”이 수식 수준에서 성립.
- (2) S_0 와 S_1 의 **가치가 같다**($20 = 20$) — “보상이 1인 행동”과 “보상이 3인 행동”을 1/2씩 골라서 **기대 즉시 보상이 2**가 되는 S_0 는, **항상** 보상 2를 받는 S_1 과 **기대 리턴이 같다**.

상태의 가치는 “기대(평균) 보상”으로 결정된다

상태의 가치는 “매 스텝 **보장되는** 보상”이 아니라 **정책이 정하는 기대(평균) 보상**에 의해 결정된다.

대조: 결정적 예($R_0 = 1, R_1 = 2$)에서는 $V(S_0) \approx 14.737 \neq 15.263 \approx V(S_1)$ 로 **두 상태의 가치가 다르다**. 같은 “서로 오가는 2상태” 구조인데도, 보상(1,2)이 **고정**되어 있어서 상태마다 **평균 보상이 달라지는** 것. 반면 이번 **확률적** 예에서는 S_0 의 **기대** 보상이 2로 **합쳐지다 보니** S_1 의 고정 보상 2와 같아 두 가치가 **동일**해진다.

주의: 두 상태의 가치가 “같다”는 이 특수한 1/2-1/2 예의 **우연**이지, 일반적으로는 다르다 — 아래 확인 문제 (a)(c)에서 정책을 0.1-0.9로 바꾸면 다시 달라진다.

손으로 한 번: 2상태 순환 MDP (결정적)

상태 2개(S_0, S_1)가 서로를 오가는 아주 작은 MDP. 정책이 고정(각 상태에서 행동 하나): S_0 에서는 항상 보상 $R_0 = 1$ 을 받고 S_1 로, S_1 에서는 항상 보상 $R_1 = 2$ 를 받고 S_0 로. $\gamma = 0.9$.

리턴의 무한 등비급수를 **직접 닫힌 형태로** 풀면:

$$V(S_0) = R_0 + \gamma R_1 + \gamma^2 R_0 + \dots = \frac{R_0 + \gamma R_1}{1 - \gamma^2} = \frac{1 + 0.9 \times 2}{1 - 0.81} \approx \mathbf{14.737}$$

$$V(S_1) = \frac{R_1 + \gamma R_0}{1 - \gamma^2} = \frac{2 + 0.9 \times 1}{1 - 0.81} \approx \mathbf{15.263}$$

벨만 기대방정식 $V(s) = R(s) + \gamma V(s')$ 을 실제로 만족하는지 **검산**:

$$R_0 + \gamma V(S_1) = 1 + 0.9 \times 15.263 \approx 14.737 = V(S_0) \checkmark$$

$$R_1 + \gamma V(S_0) = 2 + 0.9 \times 14.737 \approx 15.263 = V(S_1) \checkmark$$

정확히 들어맞는다. 이 결정적 예는 확률적 예($20 = 20$)와 대비된다: 보상(1,2)이 **고정**되어 두 상태의 **기대 보상**이 **다르니** 가치도 **다르다** (14.737 vs 15.263).

두 방식, 같은 답: 닫힌형 vs 반복 대입

- 흥미로운 확인 하나 더: 벨만방정식을 $V_0 = V_1 = 0$ 에서 시작해 **200번 반복 대입**(다음 장 정책평가 의 예고편)으로 풀어도 **정확히 같은 값**(14.737, 15.263) 으로 수렴.
- “무한급수를 닫힌 형태로 직접 계산”과 “재귀식을 값이 안 바뀔 때까지 반복 적용”이라는 서로 다른 두 방법이 같은 답에 도달한다.

이 “같은 답”은 우연이 아니다 — $\gamma < 1$ 이면 벨만방정식은 **유일한 고정점**을 갖고, 어떤 초기값에서 출발해도 그 곳으로 수렴(다음 프레임의 수축 연산 논증).

반복 대입: 재귀식을 푸는 절차 (다음 장 예고)

“값이 안 바뀔 때까지 반복 대입”이 정확히 어떤 절차인지 — 그리고 그것이 다음 장 **정책평가**의 핵심 루프인지. 벨만방정식 $V(s) = R(s) + \gamma V(s')$ 를 **가치 반복(value iteration)**으로:

```
def policy_evaluation_value_iteration(P, R, policy, gamma,
                                     max_iter=200, theta=1e-6):
    # P[s][a] = [(prob, next_state), ...]
    # R[s][a] = 즉시보상 ; policy[s] = 상태에서 s 고르는행동
    n = len(P)
    V = [0.0] * n          # 으로0 시작초기값은 ( 아무거나)
    for it in range(1, max_iter+1):
        V_new = list(V)    # 동기식: 이전""" 를V 쓴다
        delta = 0.0
        for s in range(n):
            a = policy[s]
            v_new = R[s][a] + gamma * sum(p * V[s2] for p, s2 in P[s][a])
            delta = max(delta, abs(v_new - V[s]))
            V_new[s] = v_new
        V = V_new
        if delta < theta:  # 값이거의 () 안바뀌면멈춤
            break
    return V, it
```

동기식(synchronous)으로 쓴 이유

동기식(synchronous)으로 쓴 이유

매 반복에서 **모든** 상태가 **이전** 반복의 V 를 참조해 갱신된다. 그러면 “반복을 k 번 돌렸을 때 반영된 것”이 **정확히 “미래 k 스텝까지의 리턴”**이 된다 (반복 1 = 즉시 보상만, 반복 2 = 2스텝까지, ...). 갱신을 **그 자리 바로** 하는 방식(Gauss-Seidel)도 같은 답으로 수렴하지만, “반복 횟수 = 보는 미래 스텝 수”라는 **직관이 조금 흐려진다**. 이번 절에서는 그 직관이 핵심이라 **동기식**으로 쓴다.

- 이것은 **재귀식 좌변의 $V(s)$ 를 우변으로 계속 갱신**하는 단순한 루프. 처음엔 $V = 0$ 으로 “아무것도 모른다”는 상태를 두고, 매 반복에서 “인접 상태의 (직전) 가치”를 이용해 각 상태를 갱신.
- $\gamma < 1$ 이면 이 반복은 **보증이 있다**: 모든 $V(s)$ 가 벨만방정식의 **고정점(fixed point, 좌변 = 우변인 값)**으로 수렴 — 아래 “고정점” 섹션에서 증명.

2상태 예: 반복 대입의 수렴

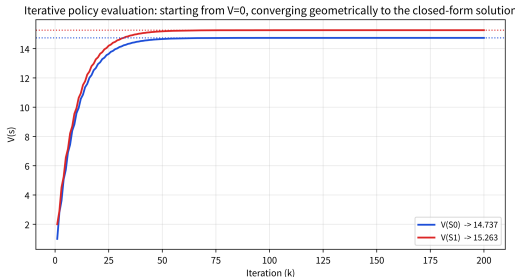
위 2상태 예($R_0 = 1, R_1 = 2, \gamma = 0.9$)를 이 함수로 $V_0 = V_1 = 0$ 에서 200번 돌리면:

반복	V_0	V_1
1	1.000	2.000 (“즉시 보상만” 반영)
10	9.598	9.941 (10스텝까지 반영)
50	14.661	15.185
200	14.737	15.263 (달힌 형태와 정확히 일치)

“반복 k 번 $\approx$ 미래 k 스텝까지의 리턴”

이 직관이 가치 반복의 **본질**. 반복을 한 번 더 하면 “한 스텝 더 먼 미래”가 반영.

γ^k 가 빠르게 0으로 줄므로(여기서 $\gamma = 0.9$, 10스텝 $0.9^{10} \approx 0.35$, 50스텝 ≈ 0.005), **수렴은 기하급수적으로 빨라진다**. 수렴 “속도”를 좌우하는 것이 바로 $\gamma - 1$ 에 가까울수록 (장기적일수록) **수렴이 느려지는** 이유이기도 하다.



반복적 정책평가 수렴 곡선 — $V = 0$ 에서 시작해
기하급수적으로 달힌해로 접근.

왜 $\gamma < 1$ 이면 수렴하는가 (증명 개요)

벨만 연산자를

$$\mathcal{B}^\pi V(s) = R(s, \pi(s)) + \gamma \mathbb{E}_{s'}[V(s')]$$

라고 쓰자. 두 값 함수 V, W 에 대해 $\gamma < 1$ 이면:

$$\|\mathcal{B}^\pi V - \mathcal{B}^\pi W\|_\infty \leq \gamma \|V - W\|_\infty$$

($\|\cdot\|_\infty$ = 최대 절댓값, “어떤 상태에서 가장 많이 다르냐”)

수축 연산 + 바나흐 고정점 정리

즉 벨만 연산자는 γ 를 계수로 줄이는 수축 (squeeze) 연산. 바나흐 고정점 정리(Banach fixed-point theorem)에 의해, 수축 연산은 고정점이 딱 하나 존재하고, 어떤 초기값에서 출발해도 그 고정점으로 수렴한다.

이 한 줄의 정리가 “가치 반복이 항상 (고유한) 답으로 수렴한다”는 **보장의 전부**.

- $\gamma = 1$ 이면 수축 상수가 1이 되어 이 보장이 **사라진다** — 수렴하지 않거나, 유한 에피소드여도 경계가 무한대가 되면 발산할 수 있다.
- 3.2절에서 “ $\gamma < 1$ 이면 수렴”이라 말한 것이, 여기서 **수학적으로 왜 그런가**가 보인다.

“벨만방정식”이라는 이름의 유래 (역사)

이 재귀식 — “현재 가치 = 지금 보상 + 할인된 다음 상태 가치” — 을 발견하고 이름을 붙인 사람이 **리처드 벨만 (Richard Bellman, 1920–1984)**.

- 1950년대 벨만은 “복잡한 최적화 문제를 작은 부분 문제로 쪼개서 그 답을 재귀적으로 조합한다”는 **동적계획법**을 체계화했고, 그 핵심 장치가 바로 이 재귀식.
- “**차원의 저주(curse of dimensionality)**”라는 용어를 만든 것으로도 유명(이번 장 머리말).

“Bellman equation”은 강화학습만의 개념이 아니다

최적제어, 금융공학, 확률과정 통계 등 “시간에 따라 상태가 변하며 **순차적으로 결정**해야 하는 문제” 전반에서 쓰이는, 20세기 수학의 **가장 널리 인용된 재귀식 중 하나**.

강화학습에서 이 방정식이 “**기대**”(expectation, 이 절) 버전과 “**최적**”(optimality, max 이 붙은, Chapter 4) 버전으로 나뉘어 등장하는 것뿐, **뼈대는 벨만이 70여 년 전에 세운 것과 정확히 같다**.

이번 절에서는 “기대” 버전($\mathbb{E}_\pi$), Chapter 4에서는 “최적” 버전($\max_a$)을 배운다.

벨만방정식의 세 가지 읽기방식

같은 방정식 $V^\pi(s) = \mathbb{E}_\pi[R_t + \gamma V^\pi(s_{t+1}) \mid s_t = s]$ 를 **세 가지 각도**로 읽을 수 있고, 이 세 가지가 각각 다른 알고리즘/관점을 낳는다:

- ① **정의(definition)로서**: “이 정책 π 의 상태가치는 다음 조건을 만족하는 **(유일한) 함수다.**”
— 이 절의 관점. V^π 를 **정의하는** 방정식.
- ② **계산 절차(procedure)로서**: “값이 안 바뀔 때까지 $V \leftarrow \mathcal{B}^\pi V$ 로 반복 대입.” — 위 정책평가 코드. **다음 장 동적계획법**의 알고리즘.
- ③ **학습 목표(learning target)로서**: “**신경망**이 이 방정식의 좌변과 우변을 같게 되도록 학습하자.” — **Chapter 9 DQN**의 손실함수. DQN의 손실 $(r + \gamma \max_{a'} Q_{\theta-}(s', a') - Q_{\theta}(s, a))^2$ 는 정확히 “**벨만방정식의 좌변 (예측값)과 우변(목표값)의 오차**”다.

가치함수(3.3) → 정책평가(Ch. 4) → 동적계획법(Ch. 4) → Q-러닝·DQN(Ch. 6, 9) → 정책 그래디언트(Ch. 10)까지, 사실상 **모든 알고리즘은 이 재귀식을 어떤 형태로 계산/학습하는가**를 다룬다. “벨만방정식을 몇 가지 버전으로 바꿔서 푸는 학기”라고 이번 학기를 요약할 수 있을 정도.

자주 하는 실수 (3.3)

실수 1: “다음 상태의 가치”에 할인율 γ 를 빼먹기

- $V(s) = R(s) + \gamma V(s')$ 처럼 γ 를 빼면, “당장의 보상”과 “아주 먼 미래의 보상”을 동등하게 취급.
- 결정적 2상태 예에서 γ 를 빼면 $V_0 = 1 + V_1$, $V_1 = 2 + V_0$ 가 되어 $V_0 - V_1 = 1$ 이면서도 $V_1 - V_0 = 2$ 가 되어 **모순(해 없음)**이 된다 — $\gamma < 1$ 이 없으면 이 단순한 2상태 예의 방정식에도 해가 없다.

실수 2: “가치함수”를 “실제 에피소드 리턴”과 혼동

- $V^\pi(s)$ 는 **기댓값**. “상태 s 에서 시작하면 리턴이 반드시 $V^\pi(s)$ 가 된다”가 **아니다**.
- “가치가 14.737인 상태”에서 10번 시작하면 10번 중 10번이 정확히 14.737이 아니라, 10번의 **평균이 대략** 14.737에 가까울 뿐. (단, 결정적 MDP 정책에서는 기댓값 = 실제값이 일치 — 위 2상태 예가 그런 경우.)

실수 3: “해는 하나”라고 단정하지 않기

- $\gamma < 1$ 이면 고정점이 **유일**. 초기값에 무관하게 같은 답으로 수렴하는 것이 **정상**.
- 만약 “초기값에 따라 다른 값으로 수렴했다”면: (1) $\gamma \geq 1$ 로 설정, (2) 코드에 할인율 곱셈이 빠져(실수 1), (3) 전이확률 합이 1이 아닌 잘못된 MDP 중 하나.
- **“초기값 무관 수렴”은 벨만방정식 코드가 맞는지 를 확인하는 간단한 건강체크.**

FAQ (1/2): V 와 Q 의 연결, 그리고 “최적”

- Q. $V^\pi(s)$ 와 $Q^\pi(s, a)$ 는 서로 어떻게 연결되나요?

- ▶ $V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a)$ — 정책이 각 행동을 고를 확률로 Q^π 를 가중평균.
- ▶ 반대로 $Q^\pi(s, a) = R(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V^\pi(s')]$ — “이번 행동만 a 로 강제로 고정하고, 그 다음부터는 π 를 따른다”.
- ▶ 두 함수는 서로를 정의하는 데 쓰이는, 동전의 양면 같은 관계.

- Q. “최적” 정책의 가치함수는 어떻게 정의하나요?

- ▶ 이번 절의 V^π, Q^π 는 주어진 정책 π 를 따랐을 때의 가치를 잰다.
- ▶ 강화학습의 궁극 목표는 “가능한 모든 정책 중 가치가 가장 높은” **최적 정책** π^* 를 찾는 것인데, 그 정의와 계산법(**벨만 최적방정식** — 이번 절의 벨만 기대방정식과 한 글자 차이지만 max 연산이 들어가 의미가 크게 달라진다)는 Chapter 4에서 바로 이어 다룬다.

FAQ (2/2): “기대”는 어디에? 왜 “풀어야” 하나?

● Q. “기대(expectation)”가 어디에 있는 건가요?

- ▶ 방정식 안의 $\mathbb{E}_\pi$ 가 바로 “기대”.
- ▶ 이 기댓값은 두 가지 무작위성에 대해 취한다: (1) **정책**이 행동을 확률적으로 고르는 무작위성 $\pi(a|s)$, (2) **환경의 전이**가 확률적으로 나타나는 무작위성 $P(s'|s, a)$.
- ▶ 결정적 MDP+결정적 정책(위 2상태 예)에서는 이 둘이 모두 **사라져** $V(s) = R(s) + \gamma V(s')$ 의 단순한 **등식**으로 축소. “기대”라는 이름은 **이 무작위성이 있을 때** 의미를 가진다.

● Q. 방정식인데 왜 “풀어야” 하나? 답이 이미 $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$ 로 쓰여 있는 것 아닌가?

- ▶ “정의”와 “계산”을 구별. $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$ 는 **무한한 경로**의 리턴을 **평균**하라는 정의 — 실제로 계산하려면 무한 경로를 모두 탐색해야 해 **계산 불가능**.
- ▶ 벨만방정식 $V(s) = R(s) + \gamma V(s')$ 는 같은 V^π 를 **인접 상태만** 쓰도록 **재귀적으로 다시 표현** — “무한 경로”를 “한 스텝”으로 압축한 이 재귀 표현이 **계산 가능**.
- ▶ 방정식을 “풀다”는 곧 이 재귀 표현을 **반복 대입(value iteration)**해 **수치적 답**을 뽑아내는 것 — **정의(무한합)와 계산(재귀식)을 연결하는 다리**.

FAQ (3/3): γ 의 극단과 정책평가

● Q. γ 가 0이면 벨만방정식은?

- ▶ $\gamma = 0$ 이면 $V^\pi(s) = R(s, \pi(s))$ — “지금 당장의 보상만”이 가치. 에이전트가 **완전히 근시안적(myopic)**이 되어 미래를 전혀 고려하지 않음.
- ▶ $\gamma = 1$ 에 가까워지면 **장기적**. γ 는 “미래를 얼마나 멀리 볼 것인가”를 조절하는 **노브(knob)** — 3.2절의 “근시안적 vs 장기적”이 수식으로 $V(s) = R(s) + \gamma V(s')$ 의 γ 항에 **정확히 대응**.

● Q. 이 절의 “반복 대입”이 Ch. 4의 “정책평가”와 관계가?

- ▶ **정확히 같은 알고리즘**. 이 절에서 “예고편”으로 간단히 보여준 반복 대입($V \leftarrow R + \gamma \mathbb{E}[V]$ 루프)은 Chapter 4 동적계획법에서 **정책평가(policy evaluation)**라고 이름 붙여, “임의의 정책 π 의 가치 V^π 를 계산하는” 표준 알고리즘으로 **정식화**.
- ▶ 또한 “이 평가된 정책을 이용해 **더 좋은 정책**을 만드는” 단계(**정책향상**, policy improvement)를 **정책평가와 교대로** 반복하는 것이 **정책반복(policy iteration)**이며, 그것이 동적계획법의 **완형**.

한 줄

이 절의 10줄 코드가 다음 장의 알고리즘 전체의 **뼈대**.

3.3절 마무리

MDP는 “지금 한 행동이 미래의 상태 자체를 바꾼다”는 사실 하나를 Chapter 2의 밴딧에 추가한 것뿐이지만, 이 한 가지 차이가 강화학습을 **훨씬 더 어렵고, 훨씬 더 흥미로운** 문제로 만든다.

- **가치함수** $V^\pi(s), Q^\pi(s, a)$ = 리턴 G_t 의 **기댓값** (상태 고정 / 상태+첫 행동 고정).
- **벨만 기대방정식** = 마르코프 성질을 “한 스텝만” 적용한 결과 — 무한합을 “지금 보상 + 할인된 다음 상태 가치”의 **재귀식**으로 압축.
- **반복 대입(정책평가)** = $\gamma < 1$ 이면 **고정점 하나로 수렴**(수축 연산 + 바나흐) — 다음 장의 알고리즘 전체의 뼈대.

3주차 정리

- ① **MDP의 다섯 요소 + 마르코프 성질 = 강화학습의 “문법”** — $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ 가 “게임의 규칙 전체”를 담고, “현재 상태만 알면 미래 분포가 결정되는가”가 성립의 잣대.
- ② **리턴 G_t 와 할인율 γ** — γ 는 수렴을 보장하는 수학 장치이자 어떤 보상을 중시하는지 정하는 설계 변수. “할인율은 발산을 막는 장치”는 반쪽이고, “할인율은 목표를 바꾼다”가 나머지 반쪽.
- ③ **가치함수와 벨만 기대방정식** — V^π, Q^π 는 “동전의 양면”이고, 벨만방정식은 **마르코프 성질을 한 스텝만 적용**한 결과. 반복 대입(정책평가)은 다음 장 동적계획법의 **빠대**.

다음 주 — Chapter 4. 동적계획법 (DP)

이번 장의 MDP를 실제로 “**풀기**” 시작한다. 전이확률 P 를 정확히 안다는 전제(모델 기반) 아래, 벨만방정식의 **고정점**을 계산하는 **정책평가**와, 최적 정책을 찾는 **정책반복**, **가치반복**을 다룬다. 이번 장은 “문제를 **정의하는** 장”이었고, Chapter 4부터는 그 정의를 “**풀어가는** 장”으로 넘어간다.